ELECTENG 209

Embedded Software Design

UART Communication

Duleepa J Thrimawithana

Learning Objectives

- Why do microcontrollers have to communicate?
- Communication protocols and categorization
- Commonly used communication protocols
- Peripheral I/O used to implement communication
- Fundamentals of UART communication
- Understanding baud rate and data rate
- What do the waveforms for a UART communication look like?
- How do we configure the UART in the ATmega328P
- What is encoding and why do we need it?
- How to develop code to implement UART communication on an ATmega328P
- How to write modular software

Lecture Quiz

- The lecture quiz is now available on Canvas
  - Quiz is available for approximately 3 hours and allows 2 attempts
  - Best of the 2 attempts taken as the final score

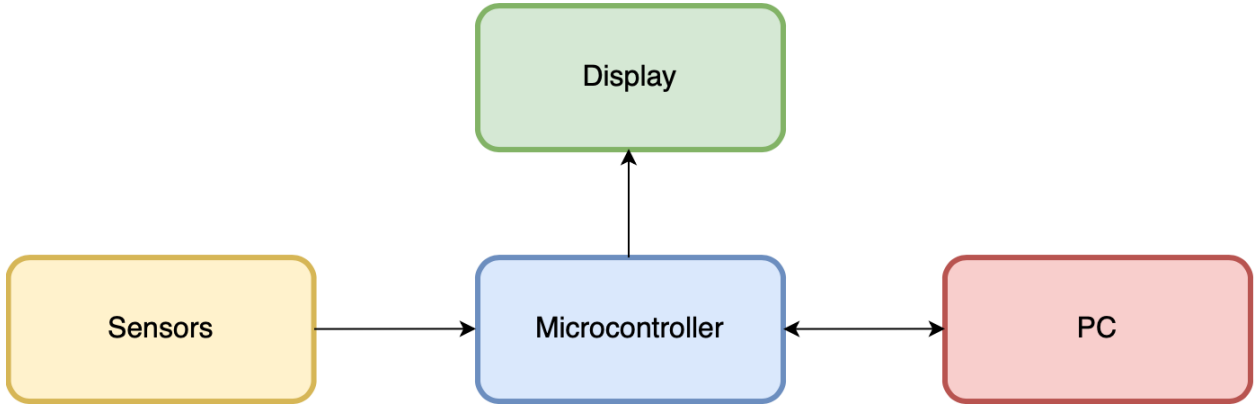




How do Microcontrollers Communicate?

Protocols and Categorization

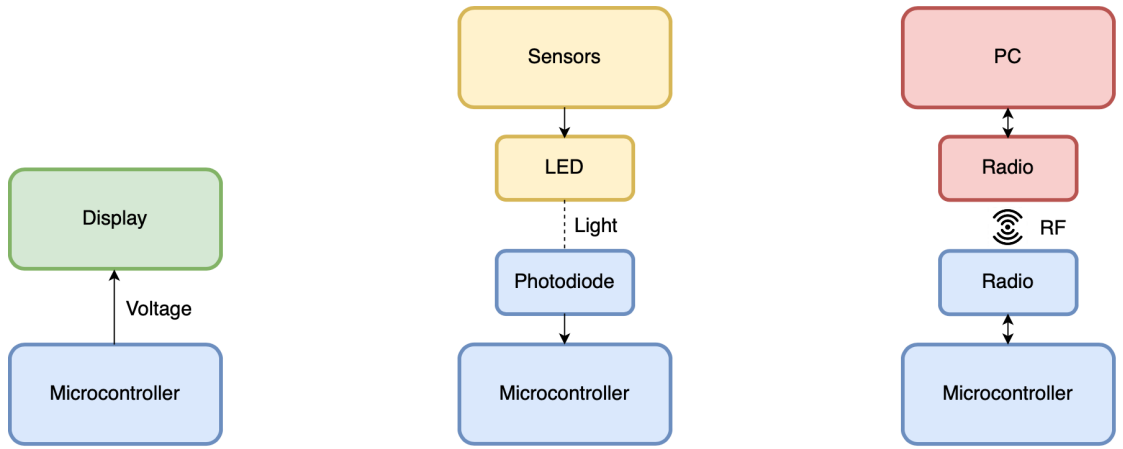
Why Communicate?



- A microcontroller seldom works in isolation
- In a typical system, a microcontroller is expected to share information with other devices around it
- To share information, microcontrollers need to communicate with other processors, modules, sensors, displays, etc.
- The communication can be for the functioning of the circuit (e.g. reading a temperature sensor), or for debugging and diagnostics (e.g. logging data to a PC)

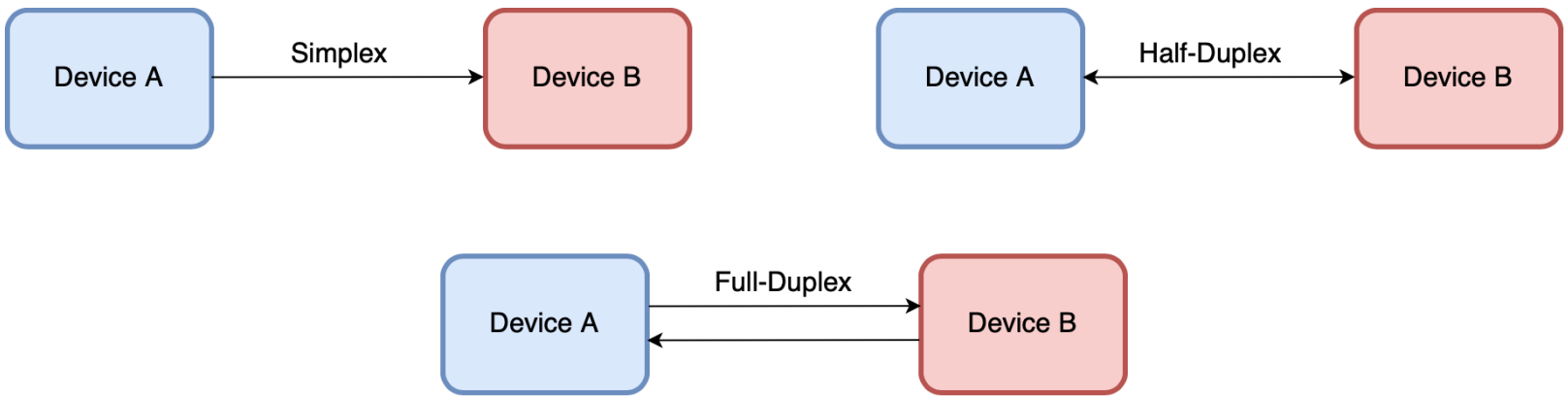
What are Communication Protocols?

- Communication with a microcontroller can be implemented in different ways
- To communicate the transmitter and receiver must agree on a specific communication protocol
  - Analogous to a spoken language, a protocol defines how information is communicated by varying a physical quantity, called the transmission medium
  - Protocols can be implemented in software or hardware
- Transmission mediums could include voltage, current, light, RF waves, acoustic waves, etc.

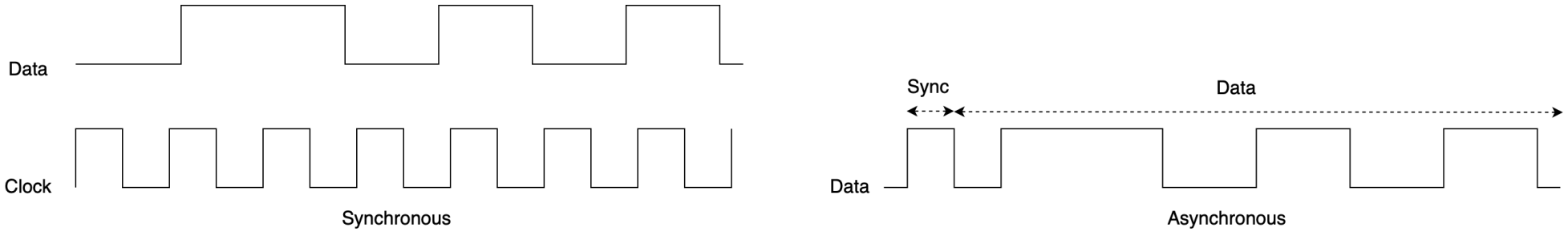


Data Transmission Modes

- Transmission modes can be broadly categorized as simplex/duplex, synchronous/asynchronous and serial/parallel
- In simplex implementations data is transferred from A to B only while in full-duplex implementations data is transferred from A to B and B to A simultaneously
  - In half-duplex implementations data is transferred from A to B or B to A, but not simultaneously
  - Duplex implementations allow bidirectional communication, but could require more channels (e.g. a wire each for send and receive)



Synchronous vs Asynchronous Transmission



- In synchronous mode a separate signal is used to indicate the start of each piece of data being transmitted
  - Often this signal is called a clock signal, like the clock used in synchronous digital circuits
  - Require more channels (e.g. separate clock wire) but have lower overhead and higher throughput
- In asynchronous mode a special “sync” event is transmitted on the data line to provide a reference point in time
  - Need to have a common understanding of time, and therefore is vulnerable to variations

Serial vs Parallel Transmission



- In serial mode only one symbol is sent at a time while in parallel multiple symbols are sent simultaneously
  - Parallel mode can send more data at a time, but require more channels (e.g. one wire per bit)
- A “symbol” is the data sent each time interval
  - In a digital (binary) system there are only two symbols, ‘0’ and ‘1’, which are referred to as bits
- There are many different communication protocols and each follow different data transmission modes
  - For example a certain protocol could implement half-duplex, synchronous, parallel communication

Common Communication Protocols

- There are many more parameters which define a communication protocol
  - We have just looked at some of the major ones
- The embedded industry has adopted a number of common communication protocols
- Serial Peripheral Interface (SPI)
  - Implements full-duplex, serial and synchronous communication
- Inter-Integrated Circuit (I2C)
  - Implements half-duplex, serial and synchronous communication
- Controller Area Network (CAN)
  - Implements half-duplex, serial and asynchronous communication
- Universal Asynchronous Receiver Transmitter (UART)
  - Implements full-duplex, serial and asynchronous communication
- Universal Serial Bus (USB)
  - Implements full or half duplex, serial and asynchronous communication



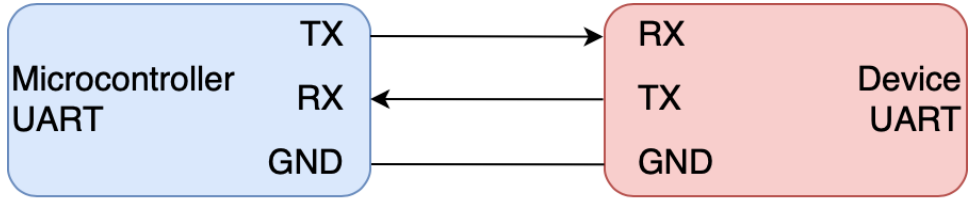


UART Communication

Fundamentals Principles

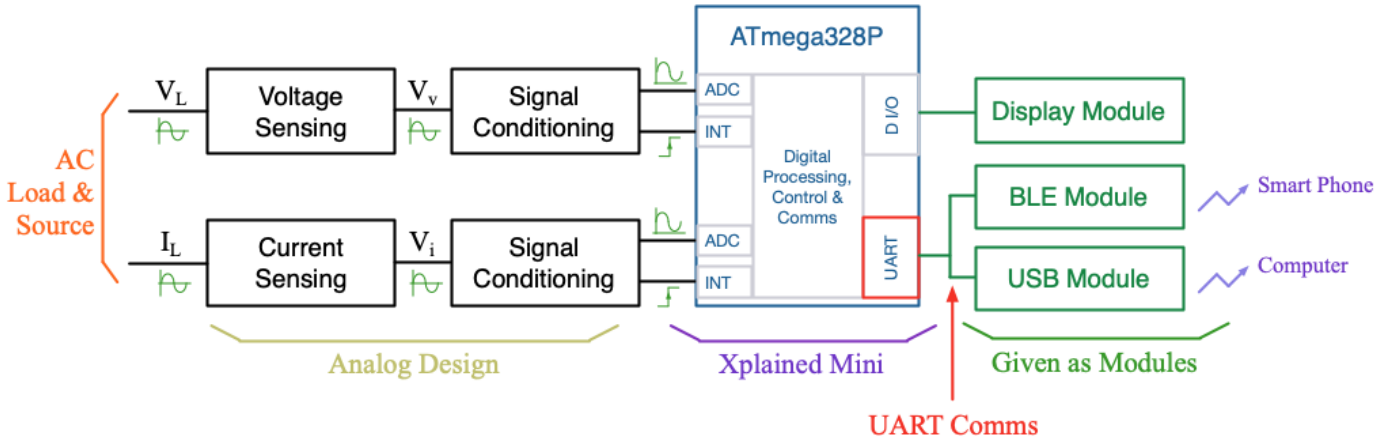
Universal Asynchronous Receiver Transmitter

- UART protocol is considered one of the simplest, cheapest and widely used
  - Commonly used for low data-rate applications like GPS, sensors, modems, etc.
- Data goes out on the transmit (TX) line and comes in on the receive (RX) line
  - The TX on microcontroller connects to RX on the device it is communicating with
  - Implements full-duplex, serial and asynchronous communication
- The microcontroller and the device are connected using 3 lines/wires (TX, RX and a ground)
  - If only needing simplex communication, 2 lines/wires are sufficient (ground, and TX/RX)
- There are extensions to the protocol which add flow control and synchronous transmission (i.e. USART)



Full Duplex UART Implementation

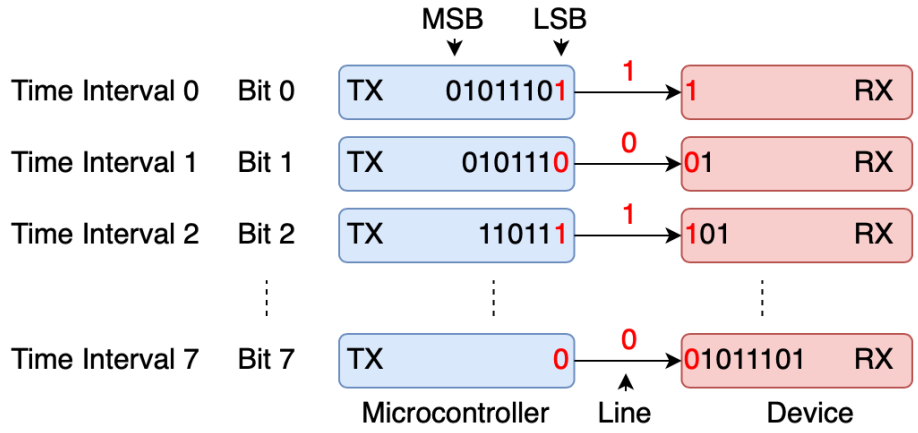
UART Communication in Your Project



- Your energy monitor need to communicate with a PC and a smart phone
  - A USB module connects the energy monitor with a PC
  - A BLE radio transceiver module is used to communication with smart phone
- Both these module communicate with the microcontroller via UART
  - Data you send to these modules will be received by the PC and smart phone in UART format
  - Received data can be viewed using a terminal program (or developing your own application)

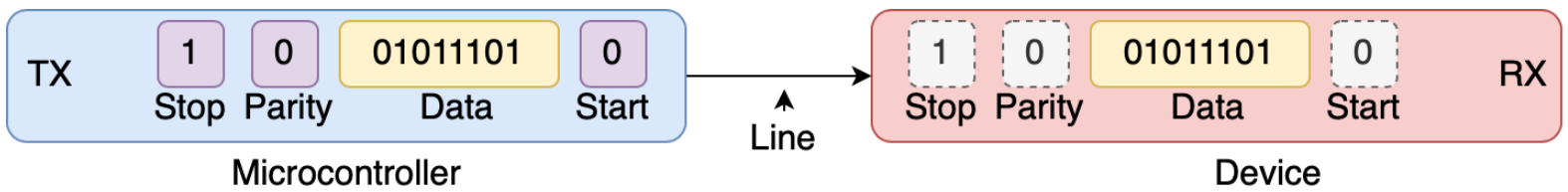
Transmitting Data using UART

- When microcontroller is transmitting data through UART, it takes a byte (8-bits) and transmits each bit at a time in a sequential fashion along the wire/line that connects its TX to RX of the device receiving the information
  - A shift registers is used to convert the byte into individual bits to send during each time interval
  - The Least-Significant-Bit (LSB) is transmitted first
- UART on the receiving device receives each bit in sequence and reassembles them into the original byte

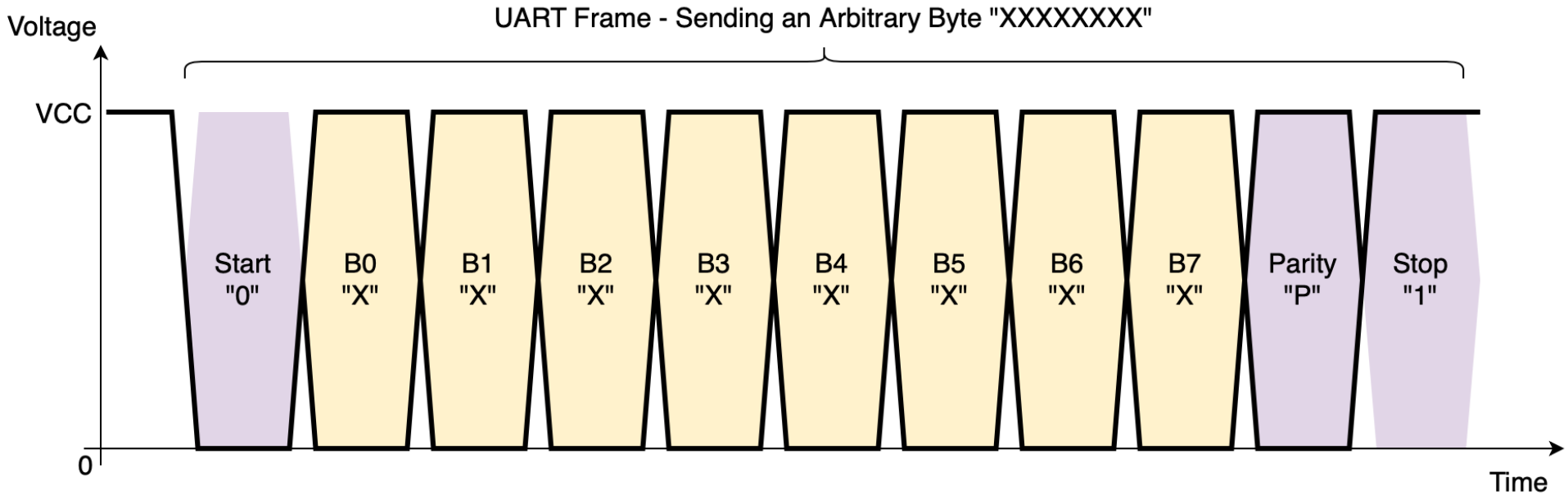


UART Frame Format

- Each data byte transmitted is preceded by a start-bit, which is indicated by a logic "0"
  - The start-bit (“sync” bit) tells the receiver when to expect a data byte from the microcontroller
- Each data byte transmitted is ended with at least one stop bit, which is indicated by a logic "1"
  - The stop bit(s) ensures the receiver can detect the next start bit
- The “idle” state (i.e. when not sending any data) of the line is logic "1"
  - This is historical from using UARTs for telegraphy
- An optional parity bit may be included before the stop-bit(s)
  - The parity bit allows the receiver to detect some instances of corruption
- “[data bits]N[stop bits] [parity type if any]” naming scheme used
  - E.g. “8N1 with odd parity” means 8 data bits are sent then an odd parity bit and 1 stop bit

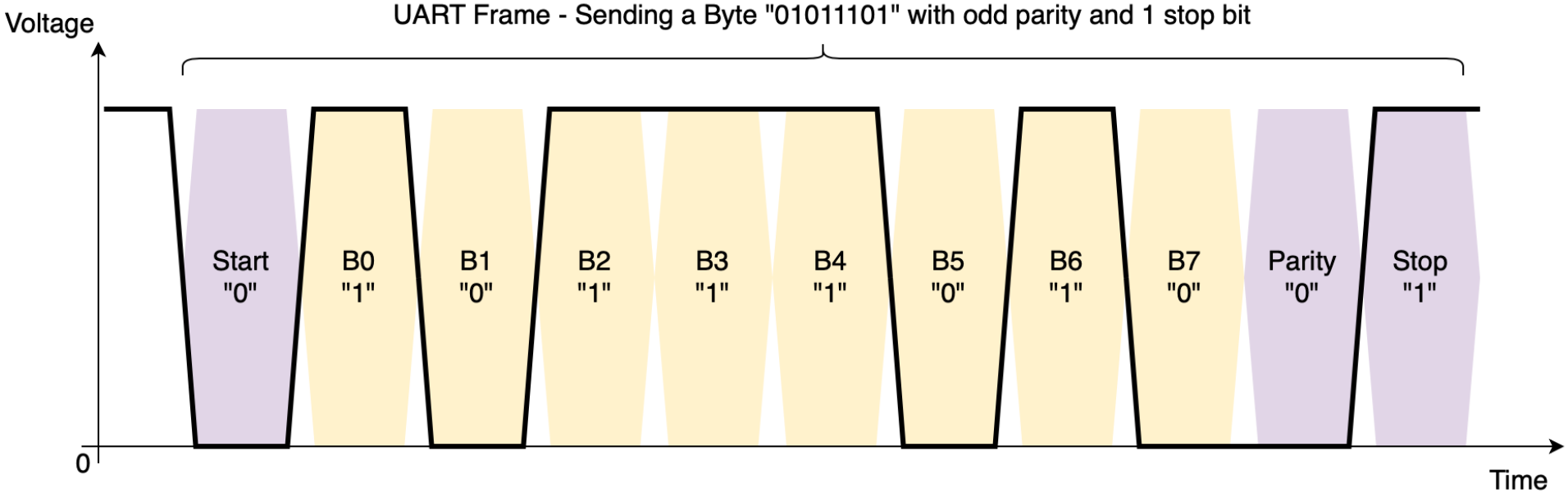


UART Waveforms (PI)



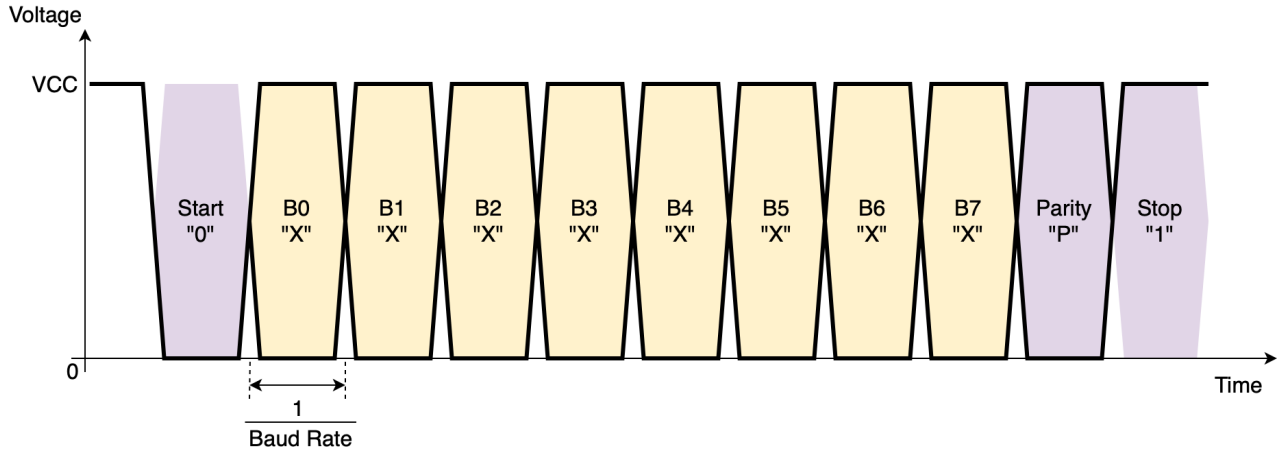
- A UART “frame” or “packet” composes of a start bit, data bits, parity bit (if used), and a stop bit (or 2)
  - Bits are represented by voltage levels VCC (for logic "1") and 0V (for logic "0")
  - Least-Significant-Bit (LSB) is transmitted first

UART Waveforms (PII)



- A UART “frame” or “packet” composes of a start bit, data bits, parity bit (if used), and a stop bit (or 2)
  - Bits are represented by voltage levels VCC (for logic "1") and 0V (for logic "0")
  - Least-Significant-Bit (LSB) is transmitted first

UART Baud Rate



- The amount of time between each bit being transmitted is determined by the baud rate

$$\text{Time per Bit} = \frac{1}{\text{Baud Rate}}$$

- Standard baud rates are 4800, 9600, 19200 etc.
  - Technically you can use any baud rate but not all are supported by a given devices

Baud Rate vs Data Rate

- The time taken to send one bit is set by the baud rate
- The total number of bits sent per frame include the start bit, data bits, parity bit (if used), and a stop bit (or 2)
- As an example, consider an 8N1 with no parity scheme
  - The UART frame would consist of 1 start bit, 8 data bits and 1 stop bit
  - Therefore, we would need to send 10 bits per frame of which only 8 are data bits
- The data rate in bits per second (bps) when using UART communication is therefore

$$\text{Data Rate} = \frac{\text{Data Bits per Frame}}{\text{Total Bits per Frame}} \times \text{Baud Rate}$$

- The time taken to transmit a certain number of data bits is therefore

$$\text{Transmission Time} = \frac{\text{Data Bits to Transmit}}{\text{Data Rate}}$$

Example: Sending 80 Bytes Over UART

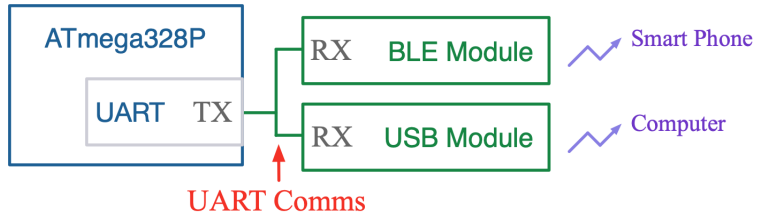
The UART of a microcontroller is setup to transmit data using 9600 baud, 8N1 with odd parity. The microcontroller transmits 80 bytes of data consecutively through this UART to a computer. Determine the following,

- Time take to send a bit in a UART frame
- Number of bits transmitted per UART frame
- Data rate in bps
- Time taken to send 80 bytes of data

UART Specifications for Project

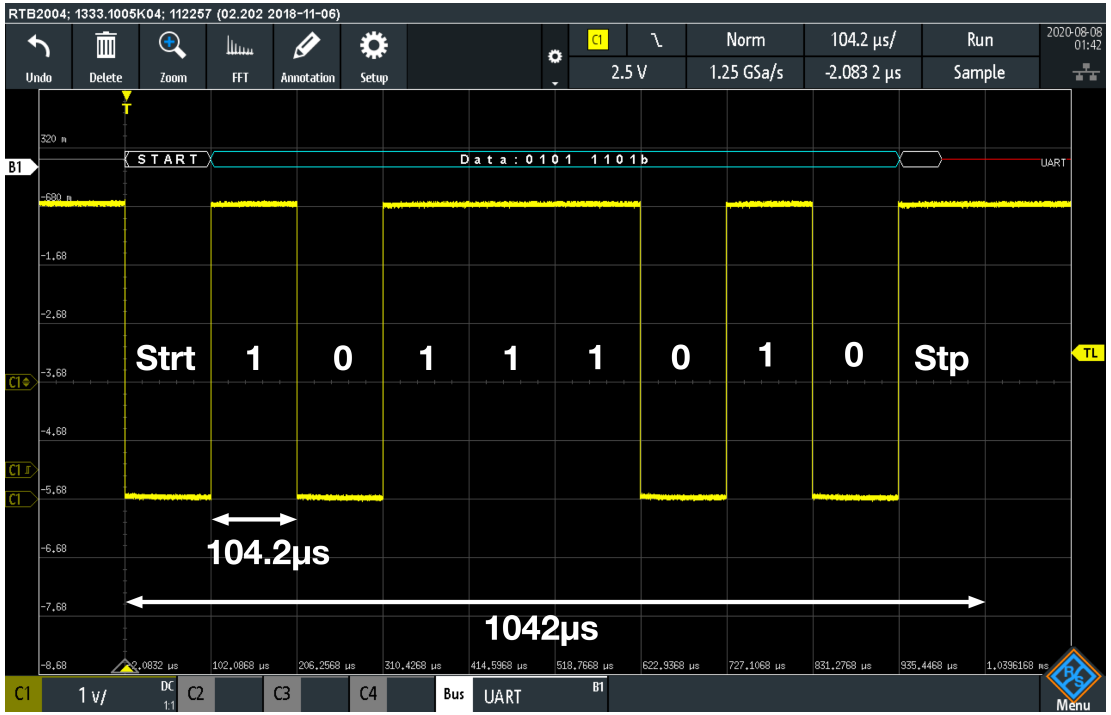
Parameter	Setting
Baud Rate	9600
Parity	None
Stop Bits	1
Transmission Mode	Asynchronous

- The HM10 BLE module used in the project only accepts 9600 baud, 8N1 with no parity
- The UART on the ATmega328P should therefore also use 9600 baud, 8N1 with no parity
- The terminal program running on PC should also be configured to use 9600 baud, 8N1 with no parity
  - The USB module uses an FTDI IC to convert the UART transmission to a USB transmission that is sent over to the PC and decoded to recover the UART transmission



UART Demonstration

- A UART transmission sent from the ATmega328P is captured using an oscilloscope
  - The ATmega328P is transmitting “01011101” (0x5D) using 9600 baud, 8N1, with no parity







The ATmega328P UART

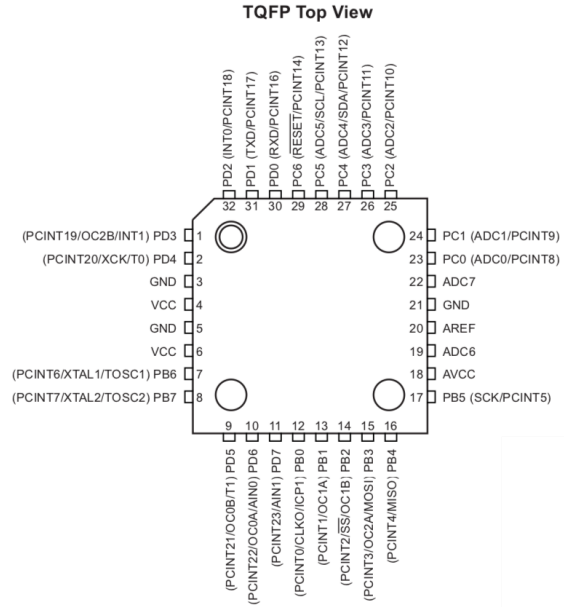
Configuring and Using

ATmega328P USART Peripheral

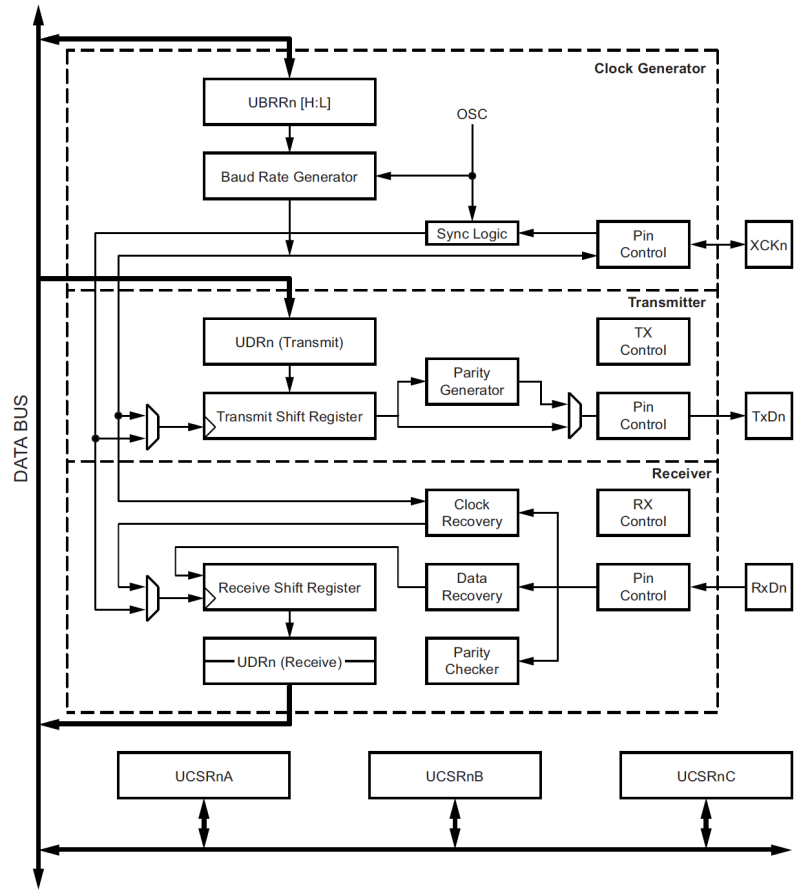
- Supports full duplex operation
- Asynchronous or synchronous communication
- Has a high resolution baud rate generator
- Supports 5, 6, 7, 8, or 9 data bits
- Supports 1 or 2 stop bits
- Odd or even parity generation and checking supported in hardware
- Data over-run and framing error detection
- Noise filtering including false start bit detection
- Transmit (TX) complete, TX ready, and Receive (RX) complete interrupts
- When using a 16MHz system clock, supports up to 1M baud

ATmega328P USART Pins

- In the ATmega328P the USART0 Receive (RX) and Transmit (TX) pins are fixed
  - RX pin is PD0 (pin 30) while TX pin is PD1 (pin 31)
- By default the bits of the DDRD register that corresponds to these two pins are configured in the correct mode (input for RX, output for TX) when we enable the UART peripheral



ATmega328P USART Implementation



ATmega328P USART Registers

- The ATmega328P only has one USART and this is named USART0
  - Since we use asynchronous mode, in lectures we will refer to USART0 as UART0
- There are 5 registers associated with UART0

Register	Functionality
UDR0	UART0 Input/Output Data Register
UCSR0A	UART0 Control and Status Register A
UCSR0B	UART0 Control and Status Register B
UCSR0C	UART0 Control and Status Register C
UBRR0	UART0 Baud Rate Register

- UDR0 register contains the UART data
- The remaining four registers are used to control the peripheral
  - Some fields are set by the user to control the operation
  - Some fields are set by the UART to indicate its current state

UDR0 Register

UDR0 (read)	RXB[7..0]
UDR0 (write)	TXB[7..0]

- The transmit data buffer (TXB) and receive data buffer (RXB) registers share the same address
  - The register accessed depends on whether a read or write is executed by the code
- When setup to transmit data
  - The code loads the data byte to transmit into the TXB register through UDR0
  - When the transmit shift register is empty the data in TXB register will be loaded automatically into the transmit shift register
  - Each bit in the transmit shift register will be automatically transmitted one at a time as we learnt previously
- When setup to receive data
  - Once the receive shift register receive a data byte, it is automatically copied into the RXB register

UCSR0A Register

Bit	7	6	5	4	3	2	1	0
UCSR0A	**RXC0**	**TXC0**	**UDRE0**	**FE0**	**DOR0**	**UPE0**	**U2C0**	**MPCM0**
Read/Write	R	R/W	R	R	R	R	R/W	R/W
Reset Value	0	0	1	0	0	0	0	0

- RXC0: *Receive Complete* flag is set when a new data frame is received
- TXC0: *Transmit Complete* flag is set after transmitting a data frame
- UDRE0: *Data Register Empty* flag when set indicates if UDR0 is ready to be loaded with new data
- FE0: *Frame Error*
- DOR0: *Data Over-Run*
- UPE0: *Parity Error*
- U2X0: *Double Transmission Speed*
- MPCM0: *Multi-Processor Communication Mode*

UCSR0B Register

Bit	7	6	5	4	3	2	1	0
UCSR0B	**RXCIE0**	**TXCIE0**	**UDRIE0**	**RXEN0**	**TXEN0**	**UCSZ02**	**RXB80**	**TXB80**
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W
Reset Value	0	0	0	0	0	0	0	0

- RXCIE0: *Receive Complete Interrupt Enable*
- TXCIE0: *Transmit Complete Interrupt Enable*
- UDRIE0: *Data Register Empty Interrupt Enable*
- RXEN0: *Receiver Enable*
- TXEN0: *Transmitter Enable*
- UCSZ02: *Character Size*
- RXB80: *Receive Data Bit 8* when using a UART frame with 9 data bits
- TXB80: *Transmit Data Bit 8* when using a UART frame with 9 data bits

UCSR0C Register

Bit	7	6	5	4	3	2	1	0
UCSR0C	**UMSEL0[1..0]**		**UPM0[1..0]**		**UBS0**	**UCSZ0[1..0]**		**UCPOL0**
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset Value	0	0	0	0	0	1	1	0

- UMSEL0[1..0]: *Mode Select* where 00 = UART, 01 = USART, 10 = Reserved, 11 = SPI
- UPM0[1..0]: *Parity Mode* where 00 = Disabled, 01 = Reserved, 10 = Even, 11 = Odd
- USBS0: *Stop Bit Select* where 0 = 1-bit, 1 = 2-bits
- UCSZ0[1..0]: *Character Size* where 00 = 5-bits, 01 = 6-bits, 10 = 7-bits, 11 = 8-bits while UCSZ02 is set to 0
- UCPOL0: *USART Clock Polarity* when operating in synchronous mode

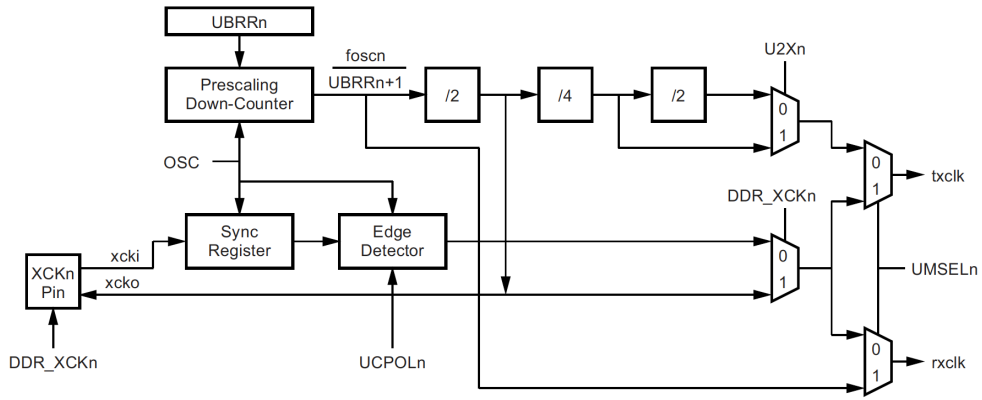
UBRR0 Register

Bit	7	6	5	4	3	2	1	0
UBRR0H	-	-	-	-	**UBRR0[11:8]**			
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R	R/W
Reset Value	0	0	0	0	0	0	0	0

Bit	7	6	5	4	3	2	1	0
UBRR0L	**UBRR0[7..0]**							
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset Value	0	0	0	0	0	0	0	0

- UBRR0[12..0]: *Baud Rate Register* value sets the baud rate by prescaling the system clock to generate a clock source for the UART

Calculating Value of UBRR0



- The UBRR0 value defines the prescaler applied to used to the system clock (f_{osc}) to obtain the UART clock (i.e. UART clock = (f_{osc} / (UBRR0 + 1))
- Under normal operation the UART clock needs to be 16 times the baud rate and therefore

$$\text{Baud Rate} = \frac{f_{osc}}{16 \times (UBRR0 + 1)} \quad \text{or} \quad UBRR0 = \frac{f_{osc}}{(\text{Baud Rate}) \times 16} - 1$$

Configuring the UART

- First we need to configure the UART as per our specifications (9600 baud, 8N1 with no parity)
  - Need to set the bits of the UCSR0A, UCSR0B, UCSR0C and UBRR0 registers
  - Have to configure the bits of the DDRD register associated with TX and RX pins (done by default)
  - Do this in an initialization function

```
// This function configures the UART to 9600 baud, 8N1 with no parity
void uart_init(uint32_t baud_rate){
  UCSR0A = 0b00000000;    //None of the settings here are used
  UCSR0B = 0b00001000;    //Set TXEN0 bit to 1 (enable transmit) and UCSZ02 bit to 0 (8 data bits)
                          //To improve readability we should aim to write this in the form "UCSR0B = (1<<TXEN0);"
  UCSR0C = 0b00000110;    //UMSEL0[1..0]=00 (UART mode), UPM0[1..0]=00 (no parity),
                          //USBS0=0 (1 stop-bit), UCSZ0[1..0]=11 (8 data bits)
  UBRR0 = 2000000 / (16 * baud_rate) - 1;
                          //Set UBRR0 as per baud rate formula
}
```

Transmitting a Data Byte

- We also know how to use the UART registers to transmit data
  - We send data by loading the data byte to be sent into the UDR0 register
  - UDR0 needs to be ready for the new data byte and this can be checked by making sure the UDRE0 bit in the UCSR0A register is set
  - We typically do the above in a function dedicated to transmitting a byte

```
// This function transmits a single byte via the UART
void uart_transmit_byte(char byte){
  while ((UCSR0A & 0b00100000) == 0){       //UDRE0 bit is checked to see if it is 0
  ;                                         //If UDRE0 bit is not 0, wait until it becomes 0
  }

  //Put the byte to be sent into the UDR0 register
  UDR0 = byte;
}
```

Transmitting an Array of Bytes

- We can also write a function that can transmit an array of bytes
  - In a loop, we need to transmit one byte at a time by calling the function we wrote to transmit a byte
  - The number of bytes in the array is determined using the method *strlen*
  - To use *strlen* you need to **#include < string.h >**

```
//This function transmits an array of bytes via the UART using uart_transmit_byte(char byte) function
void uart_transmit_array(char* msg){
    //Loop through each byte in the array
    for (uint8_t i = 0; i < strlen(msg); i++){
        uart_transmit_byte(msg[i]);                //Transmit each byte using uart_transmit_byte(char byte)
    }
}
```





Character Encoding

American Standard Code for Information Interchange (ASCII)

Why Do We Need Character Encoding?

- UART communication protocol allows us to successfully transfer data between a microcontroller and external devices
  - UART can send a single byte of data at a time
- If we only want to send numbers between 0-255 (or -128 to 127) then this works, but what if we want to send other data types?
  - For example how to send alphanumeric characters, larger numbers, key-value pairs, etc.
- We call these desired values our “character set”
- Encoding determines how our character set is represented by the data we can send
- There are many types of character encoding systems
  - ASCII, ANSI, Unicode and UTF are a few common encoding systems
- ASCII maps letters (upper and lower case), numbers (0-9), and special characters, to the range 0-127
  - For example, 0x48 is used to represent the character ‘H’

The ASCII Table

Dec	Hex	Oct	Chr	Dec	Hex	Oct	HTML	Chr	Dec	Hex	Oct	HTML	Chr	Dec	Hex	Oct	HTML	Chr
0	0	000	NULL	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SoH	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	SoTxt	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	EoTxt	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EoT	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	Enq	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	Ack	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	Bell	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	Bsp	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	HTab	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LFeed	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VTab	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FFeed	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SOut	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SIn	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAck	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	Syn	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	EoTB	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	Can	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EoM	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	Sub	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	Esc	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FSep	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GSep	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RSep	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	USep	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	Delete

charstable.com

Demo: ASCII Conversion

Completing the UART Code

```c
#define F_CPU 2000000UL                     //Defining system clock frequency

#include <avr/io.h>                         //Needed for using the macros for register addresses
#include <string.h>                         //Needed for using strlen()
#include <util/delay.h>                     //Needed for using _delay_ms()

//Prototypes of functions used
void uart_init(uint32_t baud_rate);
void uart_transmit_byte(char byte);
void uart_transmit_array(char* msg);

int main(void){

  uart_init(9600);                          //Initializing the UART to 9600 baud, 8N1, with no parity

  while (1){
    uart_transmit_byte(0x48);               //Transmitting character 'H' - alternatively uart_transmit_byte('A');
    uart_transmit_array("ello World\n\r");  //Transmitting string "ello World" with newline character '\n'
                                            //and carriage return character '\r'
    _delay_ms(1000);                        //Transmit above characters every 1s
  }
}
```

Using *printf* and *sprintf* (PI)

- The *printf* and *sprintf* commands uses ASCII encoding
  - These commands take an input string, format it, and output it
- To use *printf* and *sprintf* **#include < stdio.h >**
- We also need to redirect the output of *printf* to use the UART and to do this we bind a function which transmits a character via the UART

```
//Creating a stream variable
static FILE usart_stdout = FDEV_SETUP_STREAM(uart_printf, NULL, _FDEV_SETUP_WRITE);

int main(void)
{
  stdout = &usart_stdout;          //Binding the stream variable to stdout
  uart_init(9600);                 //Initializing the UART to 9600 baud, 8N1 with no parity

  while (1){
    printf("Hello World\r\n");     //Printing "Hello World"
    _delay_ms(1000);               //Transmit message every 1s
  }
}
```

Using *printf* and *sprintf* (PII)

- To use *printf* we need a new function called *uart_printf*

```
int uart_printf(char var, FILE *stream) {
    uart_transmit_byte(var);      //Using our original function to transmit UART data
    return 0;
}
```

- Both *printf* and *sprintf* take a format string which defines the output together with a set of arguments
- Format specifiers stating with % sign tell where to put these arguments and how to format them

```
printf("Color %s, Number %d, Hex %X","Red", 125, 125);
```

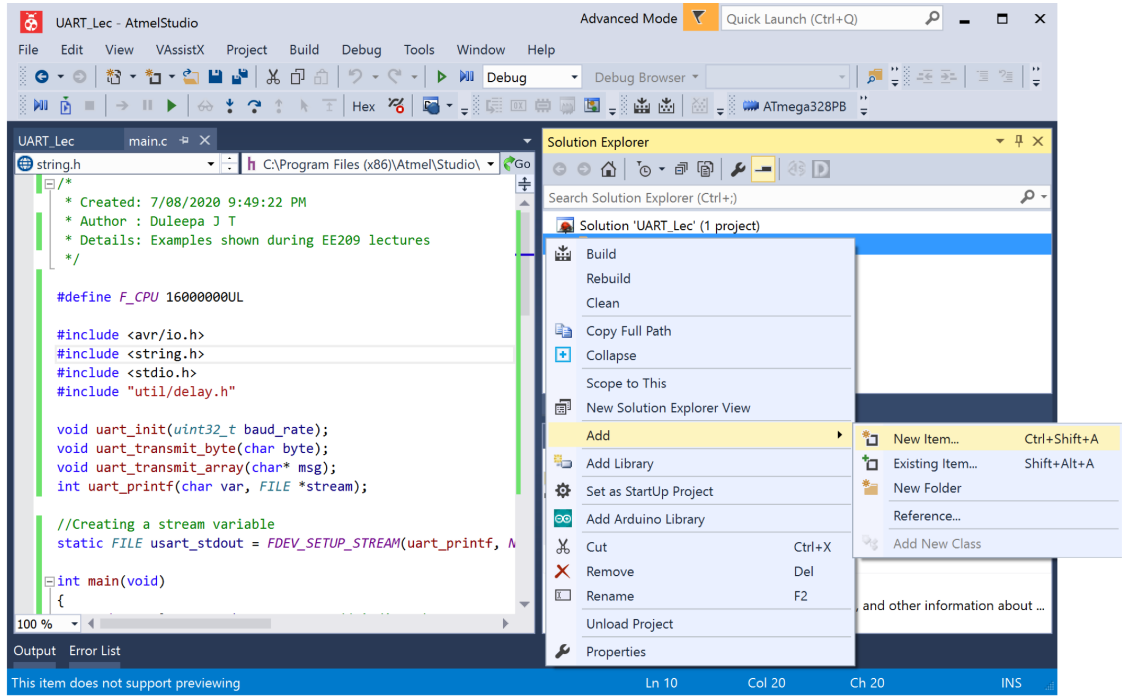
- When using sprintf the output is written into a character array
  - An example of using *sprintf* is provided in the UART lab
- The main drawback of using *printf* and *sprintf* is the overhead they add in a very resource limited environment

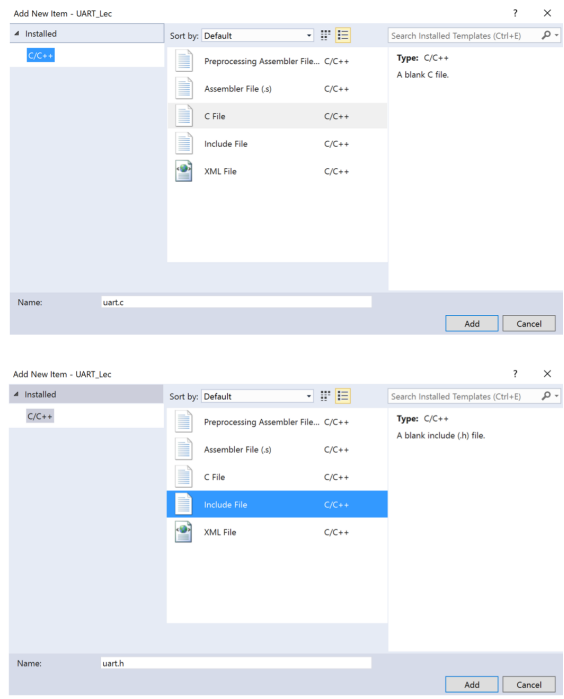
Developing Modular Software

- At this stage you should be learning to develop modular software
  - For example, all code relating each peripheral can be in a separate .c file, with a corresponding .h file
  - Allows code reuse, manageable and readable code, easy to collaborate, easy to test and validate
- As an example, lets consider the code developed to use the UART
  - Use a separate C source file called *uart.c* and a corresponding header file called *uart.h*, which will hold function prototypes for all functions in *uart.c*
  - The *uart.h* should **#include** as few other .h files as possible and it should be protected against multiple **#include**
  - **#include "uart.h"** in *uart.c* together with any necessary .h files
  - **#include "uart.h"** in *main.c* so that we can access our UART functions
- You may also want a shared .h such as *common.h* for global **#define** for example **#define F_CPU**
  - Avoid sharing global variables between .c files by declaring them as *static*

Adding Source & Header Files to Project

- To add the *uart.c* source file and *uart.h* header file to the project right-click on project name in solution explorer and click on *Add* followed by *New Item*





Things To Note

- Your transmit function is blocking and your MCU cannot do anything else while transmitting over the UART
  - It can interfere with code that has timing constraints
  - DO NOT use UART transmissions from within time critical tasks like interrupts
- You must understand examples you use for the lab or the project
  - Your individual understanding will be assessed during all assessments
  - During tests you will have to develop a C program yourself and simulate it on Proteus
  - All team members must therefore understand every part of the design
- It is critical that you complete the pre-lab of the UART lab before your lab session so that you can get help with the later parts of the lab
  - Please do not waste the opportunity to get support in the labs
  - With many students in the class, closer to due dates, there will be limited opportunities to ask for help
- You need to demonstrate UART transmission during your mid-semester review





Acknowledgments

These slides are adapted from material prepared by Travis Scott & Muhammad Nadeem